# 9주차_10장

## 비전 트랜스포머 ViT

ViT(Vision Transformer): an Image is Worth 16 × 16 Words Transformers for Image Recognition at Scale

CNN 모델의 합성곱 계층 방법을 사용하지 않고 트랜스포머 모델에서 사용되는 셀프 어텐션 적용 - 전체 이미지 한 번에 처리

물체의 크기나 해상도를 계층적으로 학습하는 스윈 트랜스포머 모델 제안됨

스윈 트랜스포머: 로컬 윈도우를 활용해 각 계층의 어텐션이 되는 패치의 크기와 개수를 다양하게 구성해 이미지 특징 학습

- 로컬 윈도우 안에 대한 어텐션, 로컬 윈도 간의 어텐션으로 수행하여 이미지 특징 계층적으로 학습시킴
- CvT 기존 합성곱 연산 과정을 ViT에 적용
- 저수준 특징과 고수준 특징을 계층적으로 반영할 수 있음
- 어텐션 연산 과정에서 쿼리(Query Q), 키(Key, K), 값(Value, V) 중 키와 값을 기존 특징 벡터보다 축소해 계산 복잡도 감소시킴

### VIT

- 이전 컴퓨터비전 관련 연구: 합성곱 신경망에 트랜스포머 모델의 셀프 어텐션 모델 적용
- VIT: 트랜스포머 구조 자체를 컴퓨터비전 분야에 적용

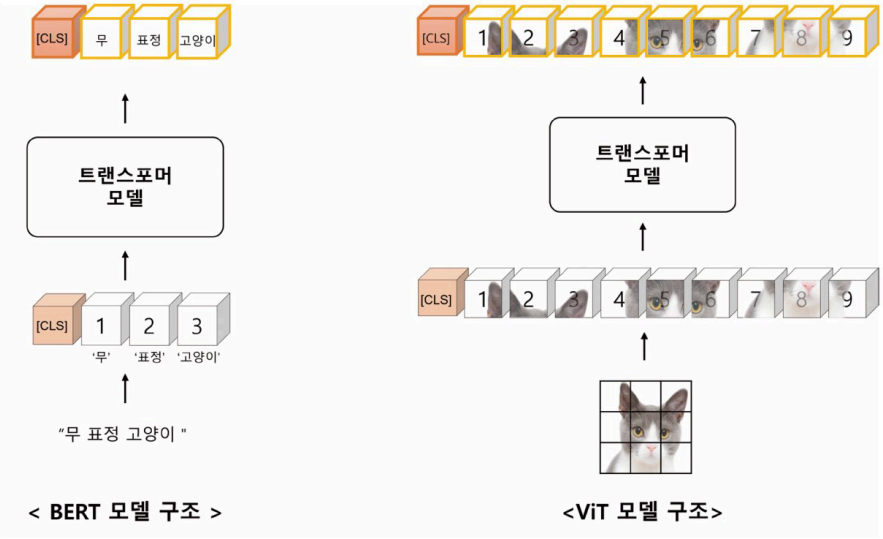



그림 10.2 BERT의 ViT 모델 구조 차이

- BERT: 문장보다 작은 단위의 토큰이 순차적으로 입력
- ViT: 이미지가 격자 단위로 작은 단위의 이미지 패치로 나뉘어 순차적으로 입력됨

### 합성곱 모델과 VIT 모델 비교

- 합성곱 신경망, 트랜스포머 모두 이미지 특징을 잘 표현하는 임베딩을 만들고자 함
- 합성곱 신경망 임베딩: 이미지 패치 중 일부만 선택해 학습 → 이미지 전체의 특징 추출
- VIT: 이미지 작은 패치들로 나누어 패치 간의 상관관계 학습 → 모든 이미지 패치가 학습에 관여; 높은 수준의 이미지 표현 제공 가능

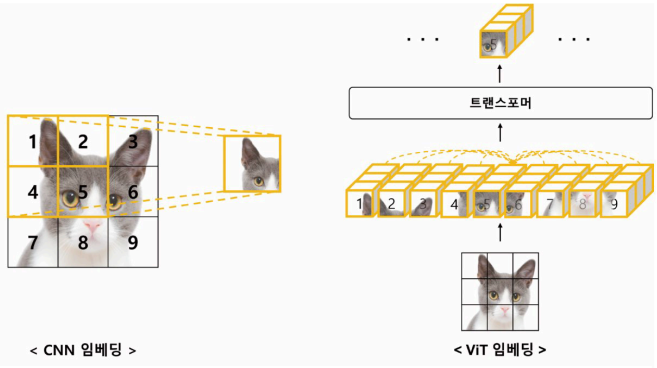



그림 10.3 합성곱 신경망과 ViT 계층의 차이

- 그림 10.3
  - 합성곱신경망의 경우 고양이 눈을 표현하는데 1, 2, 4, 5만 학습됨
  - VIT 임베딩은 셀프 어텐션 방법으로 이미지 패치가 학습에 관여함
- 좁은 수용 영역(Receptive Field, RF)을 가진 합성곱 신경망은 이미지 정보를 표현하는데 수많은 계층이 필요함
- 트랜스포머 모델은 어텐션 거리(attention distance)를 계산해 한 개의 ViT 레이어로 전체 이미지 정보 표현 가능
- ViT 패치 단위로 이미지 처리 → 작은 모델로도 높은 성능 얻을 수 있음
- ViT 입력 이미지 크기가 고정되어 있어 크기 다른 이미지 처리하려면 이미지 크기 맞추는 전처리 필요
- 합성곱 신경망은 이미지의 공간적인 위치 정보 고려
- ViT는 패치 간 상대적인 위치 정보만 고려 → 이미지 변환에 취약할 수 있음

## VIT의 귀납적 편향

- 귀납적 편향: 일반화 성능 향상을 위한 모델의 가정
  - eg. 이미지 데이터 공간적 관계 잘 표현 → 지역적 편향을 가진 합성곱 신경망에서 많이 사용

신경망 별 특징

- 합성곱 신경망
  - 이미지를 작은 조각으로 나누어 각 조각의 특징 추출 → 조합해 전체 이미지의 특징 추론
  - 지역적인 특성 강조 → 이미지 데이터에서 좋은 성능
- 순환 신경망
  - 시계열 데이터: 시간적 관계 잘 표현, sequential 편향을 가진 순환 신경망 모델 사용
  - 이전 시간의 상태 기억, 현재 입력과 결합해 다음 상태 예측
  - 순차적인 특징 강조 → 시계열 데이터에서 좋은 성능 발휘
  - 특정 관계 편향 강할수록 다른 다양한 관계 표현하기 어려움 → 귀납적 편향 약한 모델 선호
- ViT 모델: 입력 데이터의 다양한 Query, Key, Value의 임베딩 형태로 일반화된 관계를 학습 - 귀납적 편향 거의 없음, 대용량 데이터에 대해 이미지 특징들의 관계 잘 학습

표 10.1 딥러닝 모델의 귀납적 편향

| 딥러닝 모델 | 귀납적 편향 |
|---|---|
| 합성곱 신경망(CNN) | 지역적 편향 |
| 순환 신경망(RNN) | 순차적 편향 |
| ViT | 귀납적 편향이 거의 없음 |

- 귀납적 편향- 해당 모델이 가지는 구조와 매개변수들이 데이터에 적합한 가정을 하고 있음을 나타냄
- 모델은 더 적은 데이터로 높은 일반화 성능 보일 수 있음
- 너무 강한 귀납적 편향은 다른 유형의 데이터나 관계를 표현하는 데 어려움 초래할 수 있음

## VIT 모델

- 입력 이미지를 트랜스포머 구조에 맞게 일정한 크기의 패치로 나눔 → 패치를 벡터 형태로 변환하는 **패치 임베딩**과 패치와의 관계 학습하는 **인코더 계층**

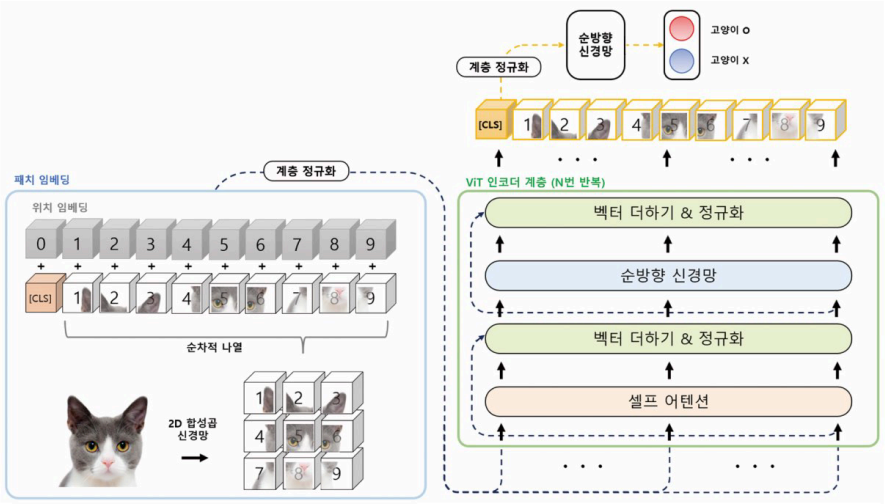



그림 10.4 ViT 모델 구조

## 패치 임베딩

- 입력 이미지를 작은 패치로 분할하는 과정
- 작은 패치로 분할하기 위해 이미지 크기 맞추는 전처리 수행되어야함

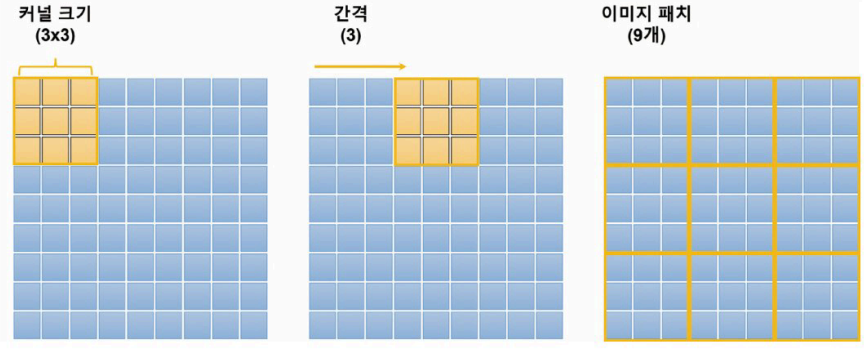



그림 10.5 이미지 패치 생성 과정

- Kernel Size, Stride 설정 - hyperparameter
- Kernel: 패치의 크기
- Stride: 패치가 이동하는 폭

$$patchsize = \frac{image\ size - kernelsize}{stride}$$

- 분할된 이미지 패치들을 배열 형태로 나열 - 왼쪽 → 오른쪽, 위 → 아래 순서로 배열

- 배열 가장 왼쪽: 분류 토큰(Special Classification Token)
- 위치 임베딩 사용해 인접한 패치 간의 관계 학습
- 패치 위치 정보 임베딩 벡터로 변환, 기존 이미지 패치 벡터들과 더함
- 계층 정규화 → 패치 임베딩
- GPU 메모리 한계 극복, 더 큰 이미지 처리 가능

### 인코더 계층

- N개의 인코더 레이어 반복적 적용
- 마지막 레이어에서 분류 토큰(특별한 패치의 특징 벡터) 추출
- 분류 토큰 벡터 - 이미지 데이터를 잘 표현하는 특징 벡터로 간주 - 전체 이미지 특징을 잘 표현하는 분류 토큰 벡터만 사용해 분류 문제 풂

### 모델 실습

```
from itertools import chain
from collections import defaultdict
from torch.utils.data import Subset
from torchvision import datasets

def subset_sampler(dataset, classes, max_len):
    # 1, 2번 수정: target_idx 선언 오타 수정 및 enumerate 올바르게 수정
    target_idx = defaultdict(list)
    for idx, (_, label) in enumerate(dataset):
        target_idx[int(label)].append(idx)

    # 3번 수정: 깔끔한 리스트 컴프리헨션
    indices = list(
        chain.from_iterable(
            [target_idx[idx][:max_len] for idx in range(len(classes))]
        )
    )
    return Subset(dataset, indices)

# 데이터셋 다운로드
train_dataset = datasets.FashionMNIST(root="./datasets", download=True, train=True)
test_dataset = datasets.FashionMNIST(root="./datasets", download=True, train=False)

classes = train_dataset.classes
class_to_idx = train_dataset.class_to_idx

print("Classes:", classes)
print("Class to Index:", class_to_idx)

# Train 및 Test Subset 생성
subset_train_dataset = subset_sampler(
    dataset=train_dataset, classes=train_dataset.classes, max_len=100
)
# 4번 수정: 테스트 셋에도 샘플러 적용
subset_test_dataset = subset_sampler(
    dataset=test_dataset, classes=test_dataset.classes, max_len=100
)

print(f"Training Data Size: {len(subset_train_dataset)}")
```

```
print(f"Testing Data Size: {len(subset_test_dataset)}")
print("First train data sample (Image tensor, Label):", train_dataset[0])
```

```
100%|████████| 26.4M/26.4M [00:01<00:00, 19.0MB/s]
100%|████████| 29.5k/29.5k [00:00<00:00, 305kB/s]
100%|████████| 4.42M/4.42M [00:00<00:00, 5.63MB/s]
100%|████████| 5.15k/5.15k [00:00<00:00, 10.9MB/s]
```

```
Classes: ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
Class to Index: {'T-shirt/top': 0, 'Trouser': 1, 'Pullover': 2, 'Dress': 3, 'Coat': 4, 'Sandal': 5, 'Shirt': 6, 'Sneaker': 7,
'Bag': 8, 'Ankle boot': 9}
Training Data Size: 1000
Testing Data Size: 1000
First train data sample (Image tensor, Label): (<PIL.Image.Image image mode=L size=28x28 at 0x7EACE5F704D0>, 9)
```

- root: FashionMNIST 데이터세트가 저장될 디렉터리 위치
- download=True - 데이터셋이 없을 경우 다운로드 됨
- Train 매개변수의 인수에 따라 훈련용 데이터세트와 테스트용 데이터세트 나눠서 불러올 수 있음
- 데이터세트에 포함된 클래스 classes 속성 통해 확인할 수 있음/ class_to_idx 속성 확인
- subset_sampler 함수 선언해 서브 샘플링 수행- dataset, classes, max_len(클래스 별 최대 샘플링 개수) 전달
- 기본값이 있는 딕셔너리(defaultdict) 클래스 활용해 클래스마다 1,000개의 데이터

```
# 10.2 이미지 전처리
import torch
from torchvision import transforms
from transformers import AutoImageProcessor

image_processor = AutoImageProcessor.from_pretrained(
    pretrained_model_name_or_path = "google/vit-base-patch16-224-in21k"
)

transform = transforms.Compose(
    [
        transforms.ToTensor(),
        transforms.Resize(
            size = (
                image_processor.size["height"],
                image_processor.size["width"]
            )
        ),
        transforms.Lambda(
            lambda x: torch.cat([x, x, x], 0)
        ),
        transforms.Normalize(
            mean = image_processor.image_mean,
            std = image_processor.image_std
        )
    ]
)

print(f"size: {image_processor.size}")
print(f"mean: {image_processor.image_mean}")
print(f"std: {image_processor.image_std}")
```

```
# 10.3 VIT 데이터로더 적용
from torch.utils.data import DataLoader

def collator(data, transform):
  images, labels = zip(*data)
  pixel_values = torch.stack([transform(image) for image in images])
  labels = torch.tensor([label for label in labels])
  return {"pixel_values": pixel_values, "labels": labels}

train_dataloader = DataLoader(
    subset_train_dataset,
```

```
    batch_size = 32,
    shuffle = True,
    collate_fn = lambda x: collator(x, transform),
    drop_last = True
)
valid_dataloader = DataLoader(
    subset_test_dataset,
    batch_size = 4,
    shuffle = True,
    collate_fn = lambda x: collator(x, transform),
    drop_last = True
)

batch = next(iter(train_dataloader))
for key, value in batch.items():
  print(f"{key}: {value.shape}")
```

```
# 예제 10.4 사전 학습된 VIT 모델
from transformers import ViTForImageClassification

model = ViTForImageClassification.from_pretrained(
    pretrained_model_name_or_path = "google/vit-base-patch16-224-in21k",
    num_labels = len(classes),
    id2label = {i: label for i, label in enumerate(classes)},
    label2id = class_to_idx,
    ignore_mismatched_sizes = True
)

print(model.classifier)
```

```
# 예제 10.5 패치 임베딩 확인
print(model.vit.embeddings)

batch = next(iter(train_dataloader))
print("image shape:", batch["pixel_values"].shape)
print("patch embeddings shape:",
      model.vit.embeddings.patch_embeddings(batch["pixel_values"]).shape
)
print("[CLS] + patch embeddings shape: ",
      model.vit.embeddings(batch["pixel_values"]).shape
      )
```

```
#10.6 하이퍼파라미터 설정
from transformers import TrainingArguments
args = TrainingArguments(
    output_dir = "../models/ViT-FashionMNIST",
    save_strategy = "epoch",
    eval_strategy = "epoch",
    learning_rate = 1e-5,
    per_device_train_batch_size = 16,
    per_device_eval_batch_size = 16,
    num_train_epochs = 3,
    weight_decay = 0.001,
    load_best_model_at_end = True,
    metric_for_best_model = "f1",
    #logging_dir="logs",
    logging_steps=125,
    remove_unused_columns=False,
    seed = 7
)
```

```
#10.7 매크로 평균 F1 점수
import evaluate
import numpy as np

def compute_metrics(eval_pred):
  metric = evaluate.load("f1")
  predictions, labels = eval_pred
  predictions = np.argmax(predictions, axis=1)
  macro_f1 = metric.compute(
      predictions = predictions,
```

```
      references = labels,
      average = "macro"
  )
  return macro_f1
```

```
# 10.8 VIT 모델 학습
import torch
import evaluate
import numpy as np
from itertools import chain
from collections import defaultdict
from torch.utils.data import Subset
from torchvision import datasets
from torchvision import transforms
from transformers import AutoImageProcessor
from transformers import ViTForImageClassification
from transformers import TrainingArguments
from transformers import Trainer


def subset_sampler(dataset, classes, max_len):
    target_idx = defaultdict(list)
    for idx, label in enumerate(dataset.targets):
        target_idx[int(label)].append(idx)

    indices = list(
        chain.from_iterable(
            [target_idx[idx][:max_len] for idx in range(len(classes))]
        )
    )
    return Subset(dataset, indices)


def model_init(classes, class_to_idx):
    model = ViTForImageClassification.from_pretrained(
        pretrained_model_name_or_path="google/vit-base-patch16-224-in21k",
        num_labels=len(classes),
        id2label={idx: label for label, idx in class_to_idx.items()},
        label2id=class_to_idx,
    )
    return model


def collator(data, transform):
    images, labels = zip(*data)
    pixel_values = torch.stack([transform(image) for image in images])
    labels = torch.tensor([label for label in labels])
    return {"pixel_values": pixel_values, "labels": labels}


def compute_metrics(eval_pred):
    metric = evaluate.load("f1")
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis=1)
    macro_f1 = metric.compute(
        predictions=predictions,
        references=labels,
        average="macro",
    )
    return macro_f1


train_dataset = datasets.FashionMNIST(
    root="../datasets", download=True, train=True
)
test_dataset = datasets.FashionMNIST(
    root="../datasets", download=True, train=False
)

classes = train_dataset.classes
class_to_idx = train_dataset.class_to_idx

subset_train_dataset = subset_sampler(
    dataset=train_dataset, classes=train_dataset.classes, max_len=1000
)
subset_test_dataset = subset_sampler(
    dataset=test_dataset, classes=test_dataset.classes, max_len=1000
)
```

```
image_processor = AutoImageProcessor.from_pretrained(
    pretrained_model_name_or_path="google/vit-base-patch16-224-in21k"
)

transform = transforms.Compose(
    [
        transforms.ToTensor(),
        transforms.Resize(
            size=(
                image_processor.size["height"],
                image_processor.size["width"],
            )
        ),
        transforms.Lambda(
            lambda x: torch.cat([x, x, x], 0)
        ),
        transforms.Normalize(
            mean=image_processor.image_mean,
            std=image_processor.image_std
        ),
    ]
)

args = TrainingArguments(
    output_dir = "../models/ViT-FashionMNIST",
    save_strategy = "epoch",
    eval_strategy = "epoch",
    learning_rate = 1e-5,
    per_device_train_batch_size = 16,
    per_device_eval_batch_size = 16,
    num_train_epochs = 3,
    weight_decay = 0.001,
    load_best_model_at_end = True,
    metric_for_best_model = "f1",
    remove_unused_columns=False,
    seed = 7
)

trainer = Trainer(
    model_init=lambda _: model_init(classes, class_to_idx),
    args=args,
    train_dataset=subset_train_dataset,
    eval_dataset=subset_test_dataset,
    data_collator=lambda x: collator(x, transform),
    compute_metrics=compute_metrics,
    processing_class=image_processor,
)
trainer.train()
```

```
#예제10.9 VIT 모델 성능 평가
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

outputs = trainer.predict(subset_test_dataset)
print(outputs)

y_true = outputs.label_ids
y_pred = outputs.predictions.argmax(axis=1)

labels = list(classes)
matrix = confusion_matrix(y_true, y_pred)
display = ConfusionMatrixDisplay(confusion_matrix=matrix, display_labels=labels)
_, ax = plt.subplots(figsize=(10, 10))
display.plot(xticks_rotation=45, ax=ax)
plt.show()
```

## Swin Transformer

- 기존 트랜스포머 기반으로 하는 모델의 문제점: 고정된 패치 크기로 인해 세밀한 예측을 필요로 하는 의미론적 분할(Semantic Segmentation)과 같은 작업에 적합하지 않음
- 트랜스포머 셀프 어텐션 : 입력 이미지 크기에 대한 2차(Quadratic) 계산 복잡도를 가짐 → 고해상도 이미지 처리하기 어려움
- ⇒ 계층적 특징 맵을 구성해 1차 계산 복잡도 갖는 스윈 트랜스포머

스윈 트랜스포머

- 이미지 분류, 객체 감지, 영상 분할과 같은 인식 작업에서 성능 좋음
- 트랜스포머 구조보다 이미지 특성 더 잘 표현
- Shifted Window 기술 - 입력 이미지 일정한 크기의 패치로 분할 - 패치 쌓아 윈도 구성/ 구성된 윈도 영역만 셀프 어텐션 계산
- 계층마다 어텐션 수행되는 패치의 크기와 개수 계층적으로 적용해 처리 → 높은 해상도와 다양한 크기를 가진 객체 효율적으로 처리

ViT와 스윈 트랜스포머 차이

- ViT:
  - 획일적인 패치 크기, 위치에 대한 제약 → 다양한 이미지 크기, 종횡비 가진 데이터 세트에 대해 적용 어려움
  - 패치 단위의 입력 받음 → 이미지의 공간 정보 완전히 보존되지 않을 수 있음
  - 이미지 패치 수 증가 → 학습 데이터 크기 증가 ⇒ 모델 학습에 필요한 계산량 증가
- 스윈 트랜스포머
  - 로컬 윈도(Local Window) 활용 → 물체의 크기, 해상도 계층적으로 학습
  - 로컬 윈도 이동하는 시프트 윈도 기술 → 입력 이미지 일정한 크기의 패치로 분할, 특징 맵 이동시켜 다음 계층에 사용 → 패치의 위치 유연하게 다룰 수 있음

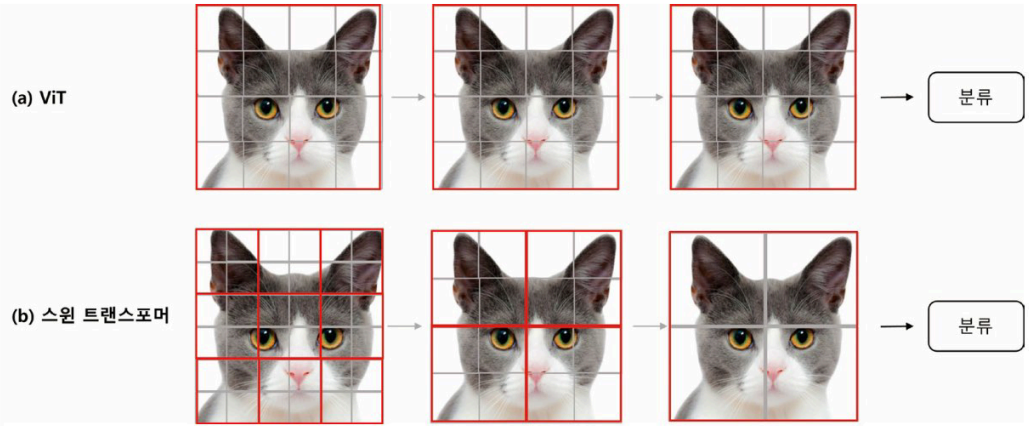



그림 10.6 ViT와 스윈 트랜스포머의 네트워크 구조

- 회색 격자: 패치, 빨간색 격자: 로컬 윈도
- ViT 네트워크 계층마다 전체 이미지 안 어텐션이 수행되는 패치 크기와 개수 동일

표 10.4 ViT와 스윈 트랜스포머 비교

| | ViT | 스윈 트랜스포머 |
|---|---|---|
| 분류 헤드 | [CLS] 토큰 사용 | 각 토큰의 평균값 사용 |
| 로컬 윈도 사용 | X | O |
| 상대적 위치 편향 사용 | X | O |
| 계층별 패치 크기 | 동일함 | 다양함 |

## 스윈트랜스포머 모델 구조

- 이미지 패치 파티션 구분 후
- 선형 임베딩(Linear Embedding), 스윈 트랜스포머 블록(Swin Transformer Block), 패치 병합(Patch Merging) 연산 반복적 수행
- 입력 이미지 일정한 크기의 패치로 분할
- 각 패치에 대해 선형 임베딩 수행
- → 패치 고정된 차원의 벡터로 변환
- 분할된 패치 기반 스윈 트랜스포머 블록 구성
  - 어텐션 계층, 계층 정규화, 다층 퍼셉트론 포함
  - 패치 간 상호작용 수행
- 패치 병합 → 전체 이미지에 대한 분류 수행
- 분할된 패치들 순차적으로 합쳐져 이미지 전체적인 정보 추출

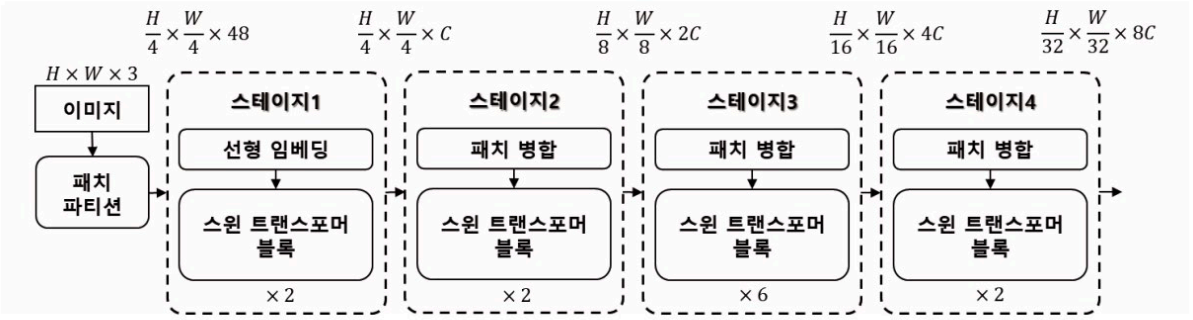



그림 10.7 스윈 트랜스포머 구조

- 스테이지 1: 선형 임베딩된 후 스윈 트랜스포머 블록 전달
- 스테이지 2, 3, 4: 패치 병합 후 각 스윈 트랜스포머 블록에 전달
  - 패치 병합: 이미지 텐서 [C, H, W]를 [2C, H/2, W/2]로 재정렬, 저차원 임베딩 수행
- 스테이지 1, 2, 3, 4 공통적으로 사용되는 스윈 트랜스포머 블록: 윈도 격자 안에 있는 패치 내부 관계, 윈도 격자 간 관계 학습 어텐션 모듈로 이뤄져 있음

## 패치 파티션 / 패치 병합

- 패치 파티션(Patch Partition): 입력 이미지 작은 사각형 패치로 분할해 처리
- 분할된 패치는 트랜스포머 계층 입력으로 사용됨

- 입력 이미지의 공간 정보 보존
- 트랜스포머 계층에 대한 계산 효율성 높임

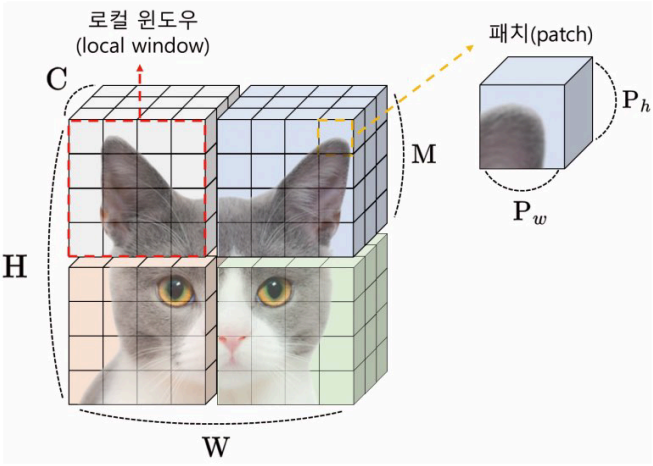



그림 10.8 패치 파티션 구조

- 일반적 이미지 텐서: [C, H, W] 구조 가짐
- 패치 파티션 통해 로컬 윈도 추가
- 패치 병합(Patch Merging): 인접한 패치들의 정보를 저차원으로 축소하는 과정
  - 이미지 [C, H, W]를 [2C, H/2, W/2]로 재정렬, 2C 차원을 저차원으로 임베딩

스윈 트랜스포머 블록

- 패치 파티션 이후 네 개 스테이지 안 반복 적용
- 스윈 트랜스포머 블록
  - 계층 정규화
  - W-MSA(Window Multi-head Self Attention)
  - MLP
  - SW-MSA

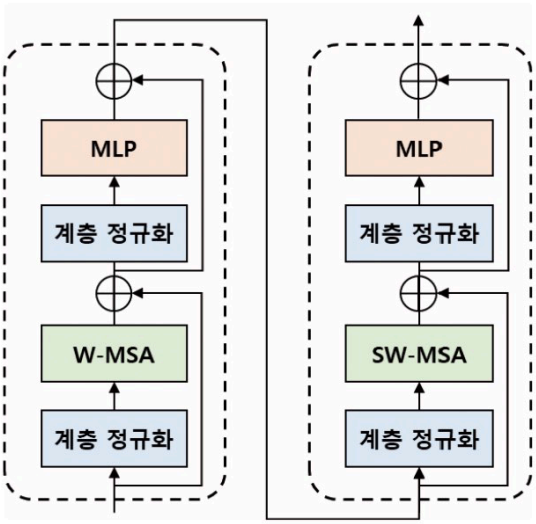



그림 10.9 스윈 트랜스포머 블록 구조

- 트랜스포머 모델 - MSA(Multi-head Self Attention) 방식
- 스윈 트랜스포머 - W-MSA, SW-MSA
  - W-MSA: 로컬 윈도 사용 멀티 헤드 셀프 어텐션
    - 입력 특징 맵을 중첩되지 않게 나눈 다음 각 윈도에서 독립적으로 셀프 어텐션 수행
    - 일정한 크기의 윈도 영역 설정
    - 해당 영역 내 픽셀 간 어텐션 계산
    - 이미지 내 전체적인 어텐션 수행 → 특정 위치에 대한 정보 추출
    - 각 윈도 내 지역적인 특징 분석
    - 윈도 내부 이미지 패치 영역에만 셀프 어텐션 수행
    - 효율적인 배치 계산(Efficient Batch Computation) → 연산량 대폭 감소
    - 로컬 윈도에서 연산을 수행하는 과정세어도 배치 축 사용 가능; 연산 속도 빨라짐
  - SW-MSA
    - 윈도 사이 연결성 구축 → 윈도 간 관계성 정보 추출
    - 윈도 가로 또는 세로 방향으로 이동, 인접한 윈도 간 셀프 어텐션 계산

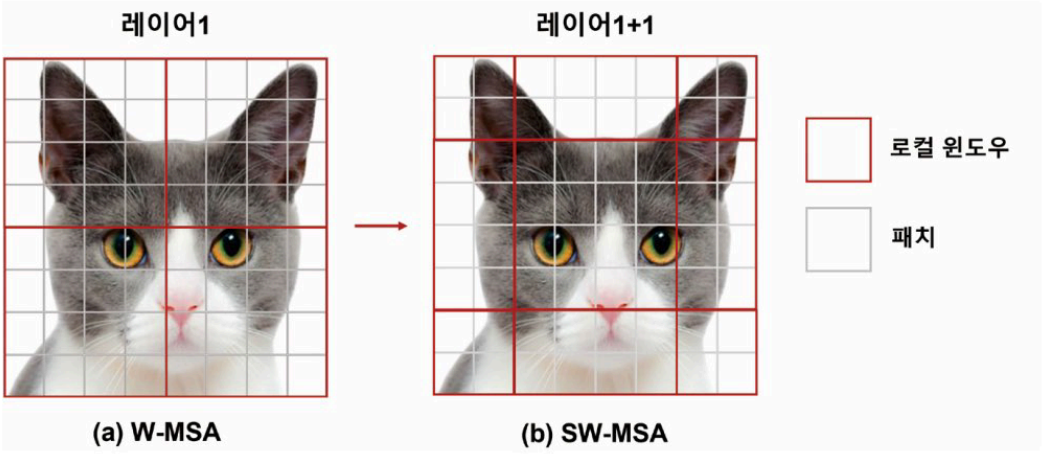



그림 10.10 W-MSA와 SW-MSA 어텐션 영역 비교

- 로컬 윈도 간 셀프 어텐션 수행해야함 - 더 많은 어텐션 연산
- 해결: 순환 시프트(Cyclic Shift), 어텐션 마스크(Attention Mask)

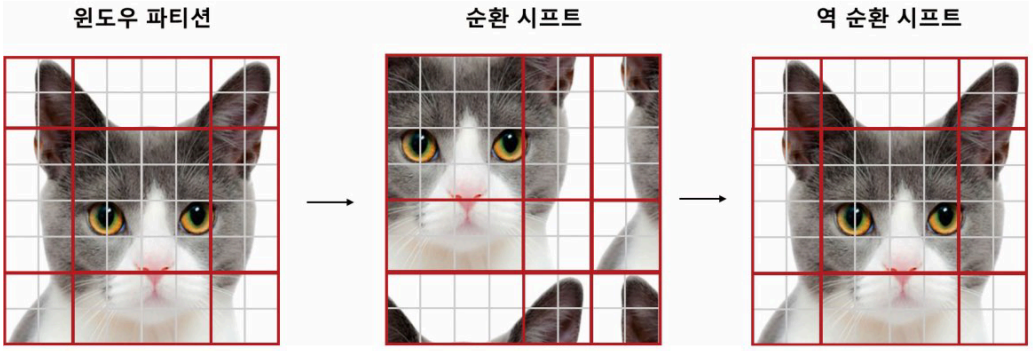



그림 10.11 순환 시프트를 활용한 SW-MSA 어텐션 계산 과정

  - 순환 시프트: 로컬 윈도 이동할 때 이동한 위치의 정보 이전 위치에서 가져옴
    - 로컬 윈도 사이즈 M보다 작은 M/2 만큼 로컬 윈도들 이동
    - 이동된 위치에서 연산 수행하지 않도록 어텐션 마스크 사용
  - 역순환 시프트(Reverse Cyclic Shift):
    - 이동된 윈도에 대한 정보 보존
    - 불필요한 연산 줄이고 효율적인 계산 가능
- 상대적 위치 편향(Relative Position Bias)
  - 로컬 윈도 안에 있는 패치 간의 상대적인 거리 임베딩




그림 10.13 상대적 위치 편향 계산 과정

- 패치 간 상대적 거리 X축과 Y축 두 가지 방식으로 표기 가능

## CvT

- CvT(Convolutional Vision Transformer): 합성곱 신경망 구조에서 활용하는 계층형 구조를 ViT에 적용
- 합성곱 신경망에서 사용하는 계층적 구조를 ViT에 적용해 이미지의 지역 특징과 전역 특징을 모두 활용, 적은 매개변수로 높은 성능 보임

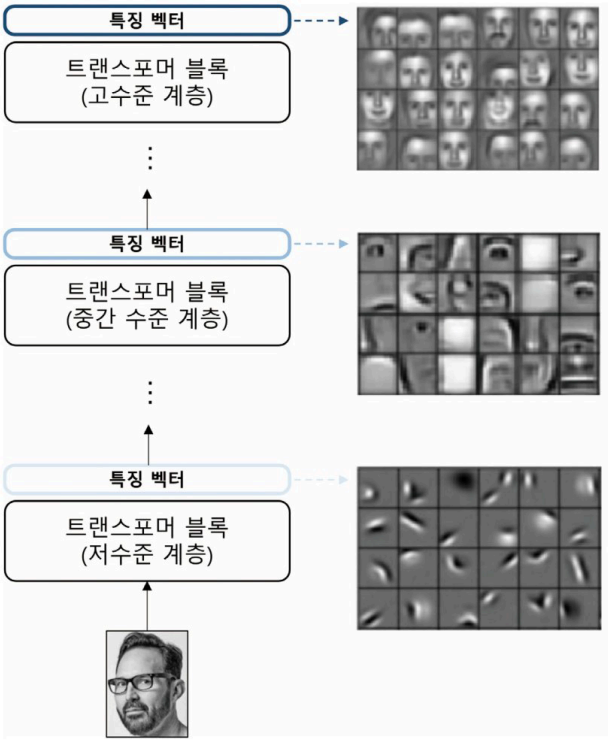



그림 10.14 CvT 모델의 계층적 특징 구조 예시

- 이미지 처리 전에 합성곱 연산 통해 특징 맵 추출
- 유의미한 패턴 추출, 일정한 크기의 패치로 나눠 트랜스포머 모델 입력으로 사용
- 패치 임베딩과 선형 임베딩을 사용하는 ViT와 달리 CvT 모델은 합성곱 연산 통해 이미지 학습

표 10.4 ViT와 CvT 모델 비교

| 모델 | 위치 임베딩 | 패치 임베딩 | 어텐션 임베딩 | 계층형 구조 |
|---|---|---|---|---|
| ViT | O | 겹치지 않는 선형 임베딩 | 선형 임베딩 | X |
| CvT | X | 겹치는 합성곱 임베딩 (overlapping) | 합성곱 임베딩 | O |

## 합성곱 토큰 임베딩

Convolutional Token Embedding

- 2D로 재구성된 토큰 맵에서 중첩 합성곱 연산 수행
- 기존 트랜스포머 모델은 임베딩 사용해 셀프 어텐션 계산, 출력 생성 - 이미지의 근본적인 2D 모양 구조 잃게 됨
- CvT 이미지의 2D 구조 보존하면서도 셀프 어텐션 계산하는 방식 사용

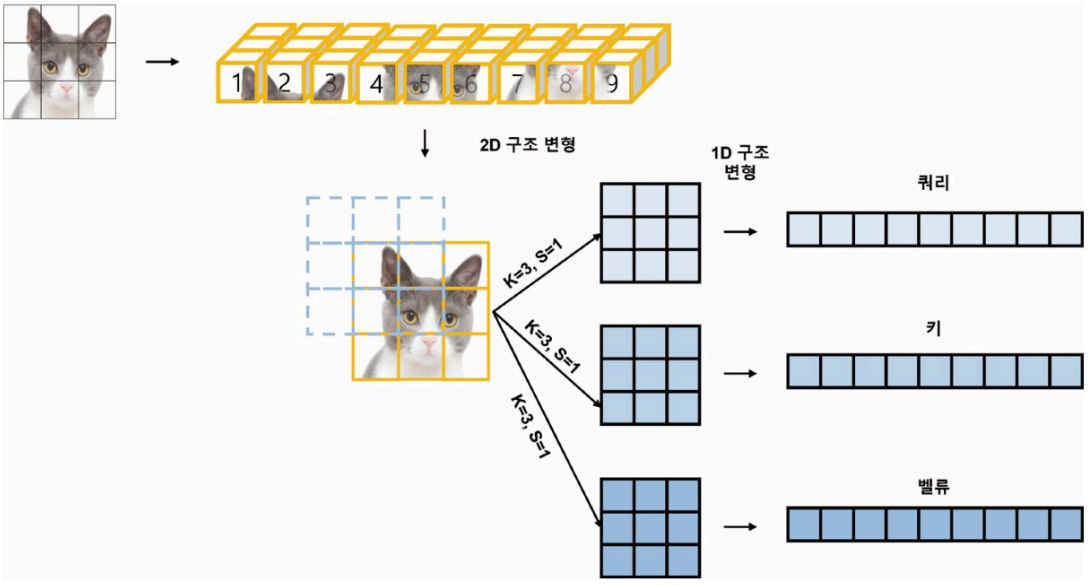



그림 10.15 2D 합성곱 기반의 어텐션 임베딩

- 각각의 패치를 쿼리, 키, 값에 대해 합성곱 연산 적용
- 1차원 구조로 재배열
- 기존 셀프 어텐션 방법으로 학습
-

## 어텐션에 대한 합성곱 임베딩

Convolutional Project for Attention

- 2D로 재구성된 토큰 맵에서 분리 가능한 합성곱 연산 적용 - 성능 저하 및 계산 복잡도 낮춤

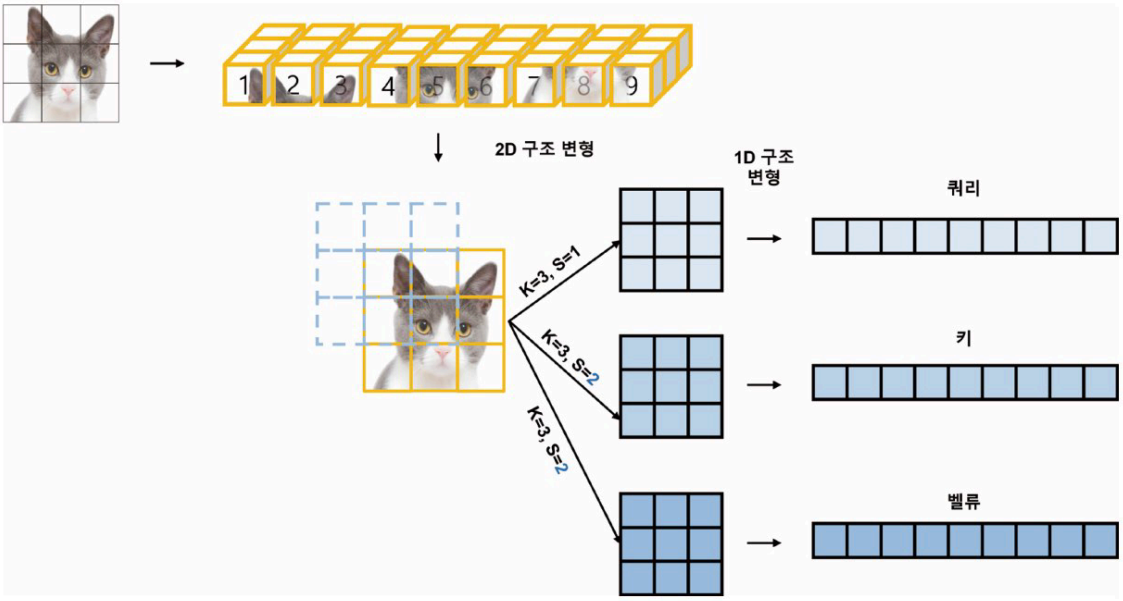



그림 10.16 2D 합성곱 기반의 축소된 어텐션 임베딩

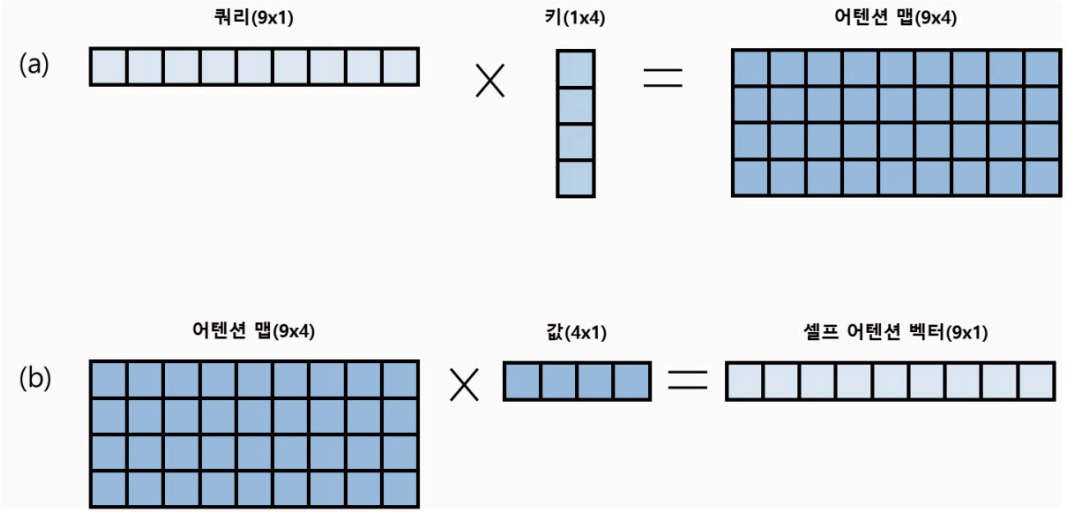



그림 10.17 축소된 셀프 어텐션 임베딩(열 × 행) 예시

- (a) 9개의 쿼리 패치 임베딩과 4개의 키 패치 임베딩에 대한 중요도 의미하는 attention map 생성
- (b) 값 패치 임베딩으로 쿼리 어텐션 맵에 대한 가중치 반영
- 입력 벡터와 출력 벡터의 패치 길이 동일
- → 계산해야하는 이미지 패치 개수가 줄어들어 연산량 감소